

1 / Explain MVC first

Code is on the left; what to say is on the right. Excerpts use ... for omitted code; long lines wrap.

Say this (20 seconds)

"My project uses API-based MVC. React is the View: it collects input and displays results. FastAPI Controllers receive requests and check authentication. Python Models apply the rules and use SQLite. Results return as JSON for React to display."

Point to these folders

View: frontend/app/ and frontend/components/

Controller: backend/app/controllers/ | Model: backend/app/models/

Flow: React form -> Controller -> Model -> SQLite -> JSON -> React

This is web MVC with a separate frontend, not server-rendered templates.

Controller: receive the request, then delegate

backend/app/controllers/rides.py | create_ride; main.py registers /api/rides

```
@router.post("")
def create_ride(body: CreateRideRequest, user_id: str =
    Depends(get_current_user_id)):
    """Driver creates a new ride with multi-stop and
    scheduling support."""
    return model.create_ride(body=body, user_id=user_id)
```

@router.post connects a POST URL to this function.

Depends runs the login check and supplies user_id.

The controller calls the model. It contains no booking SQL.

Schema: validate input, not save it

backend/app/schemas/rides.py | CreateRideRequest (excerpt)

```
class CreateRideRequest(BaseModel):
    source: str
    destination: str
    base_fare: float
    total_seats: int = 4
    scheduled_at: str | None = None
    female_only: bool = False
    stops: list[str] = []
```

BaseModel comes from Pydantic. It checks input types and supplies defaults.

Bad input produces HTTP 422. This schema is NOT the MVC Model or a database table.

Why this really separates responsibilities

Models contain rules and SQL but do not import FastAPI. Controllers handle HTTP, not SQL. React displays results; backend/app/views/ formats PDF/HTML. main.py wires the app together. Old routes/ files are compatibility imports, not a second implementation.

2 / Hotspots + smart matching

Both features: backend/app/models/rides.py. UI: rides/page.tsx and TrackingMap.tsx.

Hotspots: a list of named coordinates

backend/app/models/rides.py | HOTSPOTS (one entry, shortened)

```
HOTSPOTS = [
    {
        "id": "gate 1",
        "name": "Gate 1 (Main Entrance - Pragati Sarani)",
        "category": "campus_gate",
        "lat": 23.7745,
        "lng": 90.4255,
        ...
    },
    ...
]
```

Each place has an ID, name, category, latitude and longitude.

Add/remove places or edit coordinates here. Check ZONES aliases below the list too: some override coordinates used by matching.

The controller sends data; the View draws it

backend/app/controllers/rides.py | get_hotspots; frontend/components/TrackingMap.tsx | Marker

```
@router.get("/hotspots")
def get_hotspots():
    """Returns categorized campus pickup hotspots and transit points."""
    return model.get_hotspots()

# Model:
def get_hotspots():
    """Returns categorized campus pickup hotspots and transit points."""
    return HOTSPOTS

// React map excerpt:
<Marker key={h.id || h.name} position={[h.lat, h.lng]} icon={icon}>
```

GET /api/rides/hotspots returns the list as JSON.

React uses [lat, lng] to place each marker. The tile provider supplies the background map; it does not define our hotspots.

Smart matching: rank nearby, suitable rides

backend/app/models/rides.py | match_rides (selected lines)

```
if same_cluster or (s_coord and rs_coord and
    _haversine_km(s_coord, rs_coord) <= 0.4):
    score += 45
    reasons.append("Same Campus Pickup Zone")
...
if dist <= 1.2:
    score += 35
    reasons.append(f"Nearby Pickup ({round(dist, 1)}km)")
...
matched_results.sort(key=lambda x: (x["match_score"],
    x["scheduled_at"] or ""), reverse=True)
return matched_results
```

First remove ineligible or full rides. Then score pickup, destination, intermediate stops and time.

Same campus zone earns 45 pickup points; a pickup within 1.2 km earns 35. Higher scores come first.

`_haversine_km` measures straight-line distance on Earth, not driving distance.

Say this

"Smart matching is a rule-based scoring algorithm, not machine learning. A match score is a ranking score, not a probability or guaranteed arrival time."

3 / Multi-stop + female-only

Rules are enforced in the Model, even if someone bypasses the frontend.

Multi-stop: remember the order of stops

backend/app/models/rides.py | create_ride (stop persistence)

```
for idx, stop in enumerate(body.stops):
    if stop.strip():
        conn.execute(
            "INSERT INTO ride_stops (id, ride_id,
            sequence, place, status) VALUES (?, ?, ?, ?,
            'pending')",
            (str(uuid.uuid4()), ride_id, idx,
            stop.strip())
        )
conn.commit()
conn.close()
```

enumerate gives each stop a sequence number. ride_id links it to the ride.

SQL ? placeholders pass values separately from SQL text. commit saves changes; close releases the connection.

This is Model work: storing ride data.

A rider must get on before getting off

backend/app/models/rides.py | join_ride and update_stop_status (excerpts)

```
if pickup_route[0] >= dropoff_route[0]:
    conn.close()
    raise DomainError(status_code=400, detail="Pickup must
    be before drop-off on this ride's route")

# Allowed stop progress values:
if body.status not in ("pending", "reached", "departed"):
    raise DomainError(status_code=400, detail="Status must
    be pending, reached, or departed")
```

join_ride builds an ordered route: start, intermediate stops, destination.

It rejects unknown stops or reversed pickup/drop-off order.

Only the ride's driver can update stop progress: pending, reached or departed.

Female-only: check both creation and booking

backend/app/models/rides.py | create_ride / join_ride

```
if body.female_only and user["gender"] != "female":
    conn.close()
    raise DomainError(status_code=403, detail="Only female
    drivers can create female-only rides")

if ride["female_only"] and user["gender"] != "female":
    conn.close()
    raise DomainError(status_code=403, detail="This is a
    female-only ride")
```

Creation is rejected unless the account's stored gender is female.

Booking a female-only ride also requires a female account. Matching filters these rides too.

This uses the profile value, not independent identity verification.

If asked: what is DomainError?

It reports a model failure without importing FastAPI. controllers/errors.py converts it into an HTTP response such as 403 with {"detail": "...". The View shows that message.

4 / Scheduled booking + seat requests

Scheduling is advance booking. The driver still starts and ends the ride manually.

The frontend sends a standard timestamp

frontend/app/dashboard/rides/page.tsx | ride creation request (excerpt)

```
scheduled_at: form.scheduled_at ? new
  Date(form.scheduled_at).toISOString() : undefined,
```

The date/time input is interpreted in the browser's local timezone. `toISOString` converts it to UTC before sending it to the backend.

The Model rejects a past scheduled time

backend/app/models/rides.py | `create_ride` (selected lines)

```
parsed = dt.fromisoformat(body.scheduled_at.replace("Z",
  "+00:00"))
if parsed.tzinfo is None:
    parsed = parsed.replace(tzinfo=timezone.utc)
...
if parsed <= dt.now(timezone.utc):
    conn.close()
    raise DomainError(status_code=400, detail="Scheduled
      time must be in the future")
```

Parse the ISO timestamp; Z means UTC. Legacy values without a timezone are treated as UTC.

Reject invalid/past times with HTTP 400. A valid `scheduled_at` value is saved in rides and returned to the UI.

Confirm Seat creates a request, not a payment

backend/app/models/rides.py | `join_ride` (selected result fields)

```
"message": "Ride request sent",
"passenger_id": pid,
"status": "requested",
...
"estimated_share": projected,
"wallet_balance": balance,
"requires_topup": balance < projected,
"topup_amount": round(max(0.0, projected - balance), 2),
```

Before saving, the Model checks capacity, duplicate requests, eligibility and stop order.

`requested` means waiting for driver acceptance. Low balance returns a top-up notice; booking does not debit the wallet.

`passenger_id` here is the booking-row ID, not the user's account ID.

Demonstrate it in this order

Offer a ride with a future date/time -> find it under Scheduled Ahead -> use a second account to request a seat -> switch to the driver and accept -> start the ride manually.

Do not overclaim

Class-time scoring compares stored clock hours directly; timezone alignment there needs a fix. Advance booking itself is separate from that ranking. There is no automatic dispatch worker.

If asked: how did you check the MVC refactor?

52 backend tests passed; the 67-path API contract stayed unchanged; the frontend build passed. These were the refactor checks, not a claim that every possible bug is fixed.